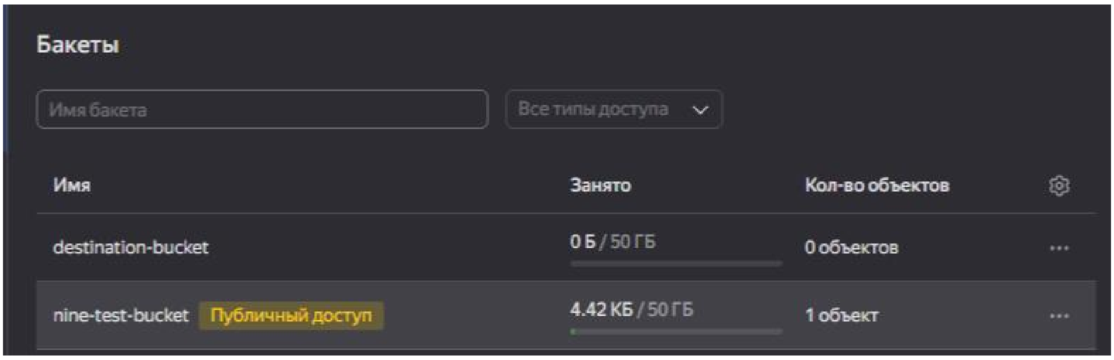
ОТЧЕТ 9

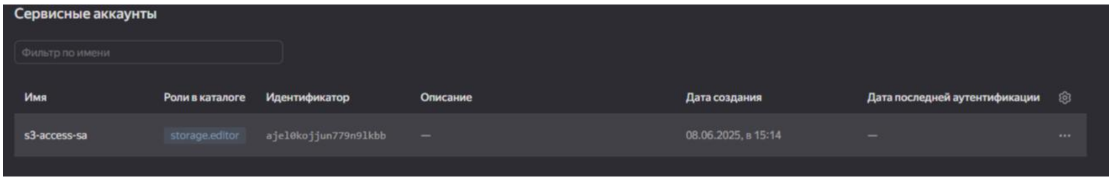
1. Создание бакета в Yandex Cloud

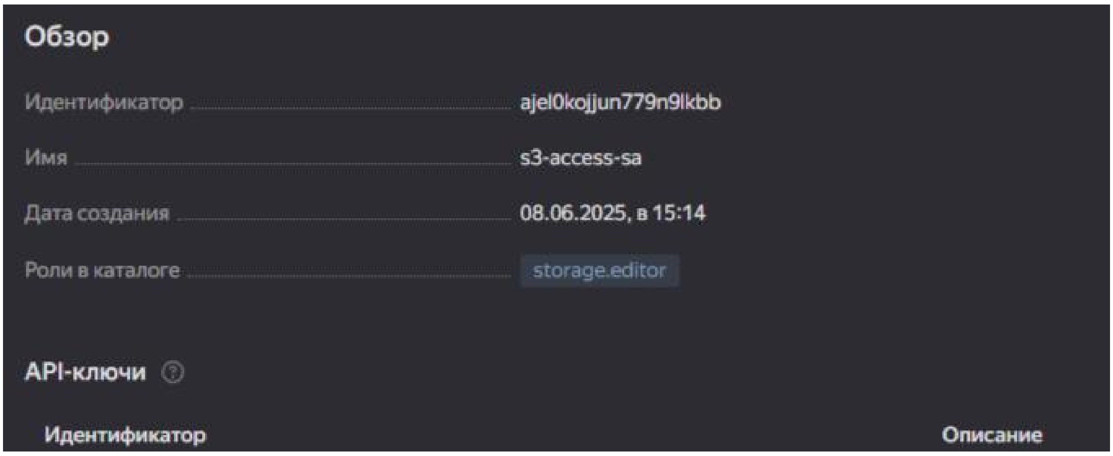
Вход в Yandex Cloud Console:
Перейдите в Yandex Cloud Console.
Откройте раздел Object Storage.
Создание бакета:
Нажмите "Создать бакет".
Укажите имя (например, nine-test-bucket).
Выберите "Стандартное хранилище" (чтобы избежать платы).
Остальные настройки оставьте по умолчанию.

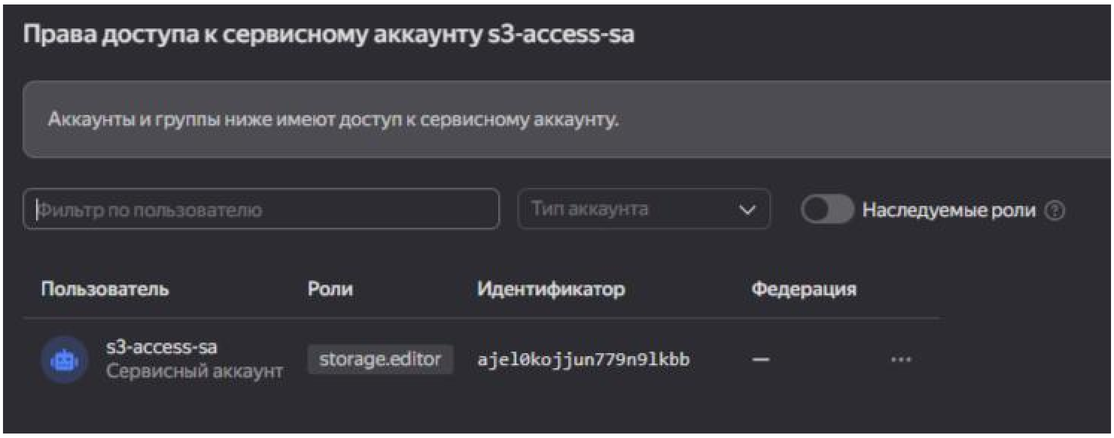


2. Создание сервисного аккаунта (IAM)

Перейдите в раздел "IAM" → "Сервисные аккаунты".
Нажмите "Создать сервисный аккаунт".
Укажите имя (например, s3-access-sa).
Назначьте роль "storage.editor" (для управления хранилищем).
Сохраните секретный ключ (JSON) — он понадобится для аутентификации.







Реализация функции (локально) для выполнения основных операций с объектами в хранилище: получение списка загруженных файлов, загрузка файла в бакет, получение файла, удаление файла.

Код программы

```
import os
import boto3
from botocore.client import Config
from botocore.exceptions import ClientError
from dotenv import load_dotenv

# Load environment variables from .env file (if it exists)
load_dotenv()

# --- Configuration ---
AWS_ACCESS_KEY_ID = os.getenv('AWS_ACCESS_KEY_ID')
AWS_SECRET_ACCESS_KEY = os.getenv('AWS_SECRET_ACCESS_KEY')
REGION_NAME = os.getenv('REGION_NAME')
ENDPOINT_URL = os.getenv('ENDPOINT_URL')
BUCKET_NAME = os.getenv('BUCKET_NAME')

#Check that all values exits or exit
if not all([AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, REGION_NAME, ENDPOINT_URL,
BUCKET_NAME]):
    print ("Make sure you .env file is properly setup!")
    exit()

# ---   S3 Client Initialization ---
try:
    s3 = boto3.client(
        's3',
        endpoint_url=ENDPOINT_URL,
        aws_access_key_id=AWS_ACCESS_KEY_ID,
        aws_secret_access_key=AWS_SECRET_ACCESS_KEY,
        region_name=REGION_NAME,
        config=Config(signature_version='s3v4', s3={'addressing_style': 'virtual'}),
        use_ssl=False,   # Set to True if you're using HTTPS for MinIO (and have a certificate)
        verify=False   # Set to True if you're using HTTPS for MinIO (and have a certificate)

    )
    print("S3 client initialized successfully.")
except ClientError as e:
    print(f"Error initializing S3 client: {e}")
    exit(1)   # Exit if the client cannot initialize

# --- Function Definitions ---

def upload_file(file_path, object_name=None):
    """Uploads a file to the bucket."""
    if object_name is None:
        object_name = os.path.basename(file_path)
```

```
    try:
        with open(file_path, 'rb') as f:    # Open in binary mode
            s3.upload_fileobj(f, BUCKET_NAME, object_name)

        print(f"File '{file_path}' uploaded to '{object_name}' successfully.")
        return True

    except FileNotFoundError:
        print(f"Error: File '{file_path}' not found.")
        return False

    except ClientError as e:
        print(f"Error uploading file: {e}") #Removed detail error which is useless
        return False

def list_files():
    """Lists all files in the bucket."""
    try:
        response = s3.list_objects_v2(Bucket=BUCKET_NAME)

        if 'Contents' in response:
            print("Files in bucket:")
            for obj in response['Contents']:
                print(f"- {obj['Key']}")
            return [obj['Key'] for obj in response['Contents']]
        else:
            print("Bucket is empty.")
            return []

    except ClientError as e:
        print(f"Error listing files: {e}") #Removed detail error which is useless
        return None


def download_file(object_name, local_path=None):
    """Downloads a file from the bucket."""
    if local_path is None:
        local_path = object_name

    try:
        s3.download_file(BUCKET_NAME, object_name, local_path)
        print(f"File '{object_name}' downloaded to '{local_path}' successfully.")
        return True

    except ClientError as e:
        print(f"Error downloading file: {e}")#Removed detail error which is useless
        return False

def delete_file(object_name):
    """Deletes a file from the bucket."""
    try:
        s3.delete_object(Bucket=BUCKET_NAME, Key=object_name)
        print(f"File '{object_name}' deleted successfully.")
        return True

    except ClientError as e:
        print(f"Error deleting file: {e}")#Removed detail error which is useless
        return False

# --- Main ---

if __name__ == "__main__":
    TEST_FILE = "test_file.txt"

    # Create a test file
```

```
    try:
        with open(TEST_FILE, "w", encoding="utf-8") as f:
            f.write("This is a test file for Yandex Object Storage.")
    except Exception as e:
        print(f"Error creating test file: {e}")
        exit(1)

    # Demonstrate operations
    print("\n1. Uploading file...")
    upload_file(TEST_FILE)

    print("\n2. Listing files...")
    list_files_list = list_files()

    print("\n3. Downloading file...")
    DOWNLOADED_FILE = "downloaded_" + TEST_FILE
    download_file(TEST_FILE, DOWNLOADED_FILE)

    print("\n4. Deleting file...")
    delete_file(TEST_FILE)

    print("\n5. Listing files after deletion...")
    list_files()

    # Clean up local files
    try:
        os.remove(TEST_FILE)
        os.remove(DOWNLOADED_FILE)
    except FileNotFoundError:
        pass #don't throw an error if one does not exist

    print("\n--- Done ---")
```

Почему-то не удается решить проблему не смотря на то что все правильно сделано и все файлы созданы и введены верно ключи